Table of Contents 目 录

Table of contents for the figure 图目录.....2
1 简介.....4
1.1 目的 4
1.2 当前的局限性.....4
1.3 挑战与需求.....4
2 算法背景.....4
2.1 传统 RAG 与 GraphRAG 的演进脉络.....4
2.2 专业文档的结构化解析需求.....4
2.3 可解释检索与证据覆盖的重要性.....4
3 当前业界研究进展.....5
3.1 知识图谱增强检索的发展方向.....5
3.2 多模态文档解析与结构化索引进展.....5
3.3 可解释问答与工作台化趋势.....5
4 算法设计与实现.....6
4.1 关键技术与创新.....6
4.2 算法原理及技术实现.....6
4.2.1 底层结构化 ingestion pipeline.....6
4.2.2 Entity-Relation-Chunk 异构证据图与混合检索.....6
4.2.3 kg-workbench 可视化与可解释问答工作台.....6
5 实验结果及解读.....7
5.1 实验条件与数据准备.....7
5.2 测试数据.....7
5.2.1 代码与提交演进样本.....7
5.2.2 benchmark、trace 与样例项目.....7
5.2.3 工作台截图与交互效果.....8
5.3 测试步骤及结果.....11
5.3.1 代码验证结果.....11
5.3.2 结构化检索与问答效果分析.....11
5.3.3 工作台联动展示效果.....11
5.4 结论 11
6 引用.....12
7 附录.....12
7.1 关键模块与证据清单.....12
7.1.1 关键代码、命令与输出文件.....12

Table of contents for the table 表目录

表格 1 ragent 与 kg-workbench 的关键结构升级 5
表格 2 代码验证、benchmark 与 trace 实证摘要 10

Table of contents for the figure 图目录

图 1 kg-workbench 全局图谱与问答工作台主界面 9
图 2 kg-workbench 图谱、证据面板与推理链路联动界面 10

All rights reserved 版权所有，侵权必究

Abbreviations 缩略词

Key words 关键词：结构化证据检索；知识图谱增强检索；Entity-Relation-Chunk；多模态文档解析；可解释问答；可视化工作台

Abstract 摘要：本报告基于对 /Users/liguowei/ubuntu/ragent 与 /Users/liguowei/ubuntu/kg-workbench 两个本地仓库的代码、提交演进、样例数据、基准评测、trace 记录和工作台截图的系统审阅，分析项目如何从传统“文本块向量检索 RAG”升级为“Entity-Relation-Chunk 异构证据图驱动的结构化证据检索系统”。其中，ragent 负责专业文档解析、增强 Markdown 构建、实体关系抽取、图谱构建、混合召回、重排序与证据覆盖选择；kg-workbench 则把这些底层能力转化为可视化图谱浏览、证据追溯、问答会话管理和推理链路展示能力。报告同时结合真实 benchmark、命令校验和 trace 样例说明系统已经从 demo 型 RAG 演进为可追溯、可调试、可交互的专业文档 KG-RAG 系统。

List of abbreviations 缩略语清单：

Abbreviations 缩略语	Full spelling 英文全名	Chinese explanation 中文解释
RAG	Retrieval-Augmented Generation	检索增强生成
KG	Knowledge Graph	知识图谱
ERC	Entity-Relation-Chunk	实体-关系-证据块异构图
AST	Abstract Syntax Tree	抽象语法树/Markdown 结构树
VDB	Vector Database	向量数据库/向量索引

All rights reserved 版权所有，侵权必究

1 简介

1.1 目的

本报告的目标不是泛泛介绍 RAG，而是基于仓库代码、提交历史、样例数据与实际命令结果，客观总结 ragent 与 kg-workbench 组成的整体系统如何完成架构升级。报告重点回答三个问题：一是 ragent 在算法与工程层面到底做了哪些可落地的增强；二是这些增强如何把系统从传统 chunk 检索推向结构化证据检索；三是 kg-workbench 如何把底层 KG-RAG 能力变成可观察、可交互、可验证的工作台。

因此，本文的叙事主线不是“项目做了很多功能”，而是“项目如何围绕专业文档问答中的证据结构化、复杂检索和可解释性问题，形成自底向上的完整技术闭环”。

1.2 当前的局限性

传统文本块向量检索 RAG 的基本流程是切 chunk、做 embedding、向量召回、把若干片段拼接给大模型。这个范式在开放文本或简单 FAQ 场景中可用，但在专业 PDF、标准文档、医学营养指南和多模态技术资料中会迅速暴露局限：图表与图片说明难以纳入统一索引，表格结构被打散后语义失真，实体与关系缺乏显式连接，查询拆解和多跳证据聚合能力较弱，最终答案虽然能“引用文本”，但很难说明为什么是这些证据而不是别的证据。

从工程视角看，传统 RAG 还常常缺少 ingestion 状态管理、失败回滚、可复用 trace、宽表导入、多 provider 模型接入和工作台式调试界面，因此在面向真实业务文档时容易停留在演示级能力。

1.3 挑战与需求

专业文档知识问答需要的不只是更高的召回率，而是更强的结构保持能力、来源可追溯能力和交互分析能力。系统既要保留章节、页码、表格行、图片描述和原文 chunk 之间的对应关系，也要支持实体关系抽取、图谱扩散、query variant 拆解、重排序和证据覆盖选择，从而回答复合问题、约束问题和带计算逻辑的问题。

同时，系统还需要把检索过程本身暴露给使用者：不仅输出答案，还能回放关键词提取、图谱命中、混合召回、rerank 和最终证据选择链路，这正是 kg-workbench 作为上层工作台存在的必要性。

2 算法背景

2.1 传统 RAG 与 GraphRAG 的演进脉络

RAG 的基本思想是将外部知识检索与生成模型结合，通过非参数化记忆弥补参数化模型的知识边界。GraphRAG 进一步强调：当问题不只是查找相似片段，而是要求跨段落整合、主题归纳或关系推理时，单纯的向量近邻难以稳定支撑答案，图结构、实体关系和社区聚合会成为更有效的中间表示。

ragent 的设计并没有照搬某一篇 GraphRAG 论文，而是围绕专业文档问答场景，把图谱增强能力下沉到“实体-关系-原文 chunk”的证据级组织方式上，使图谱与来源证据始终绑定。

2.2 专业文档的结构化解析需求

对于 PDF、标准规范、营养指南等文档，真正决定检索质量的第一步往往不是向量模型，而是解析质量。章节层级、页码、表格、图片、图注、宽表记录和增强 Markdown 的保真程度，直接决定后续切块是否还能保留语义边界。

因此，ragent 把 MinerU 解析、Markdown AST 切块、表格行级拆分、图片说明注入和内容块页码映射放在核心链路前部，先把“可检索对象”变成结构化证据单元，再进入图谱和向量层。

2.3 可解释检索与证据覆盖的重要性

All rights reserved 版权所有，侵权必究

复杂问题往往包含多个子意图，如人群条件、数值阈值、排序要求和依据说明。若系统只按整体 query 做单路召回，容易得到若干高相似度但覆盖不完整的片段。可解释检索的关键不是“展示更多日志”，而是让检索过程本身围绕覆盖率、来源多样性、图谱支持度和低信号过滤等原则组织。
这也是 ragent 后续引入 query variant 拆解、向量-图谱联合打分、rerank 和最终证据覆盖选择逻辑的理论背景。

3 当前业界研究进展

3.1 知识图谱增强检索的发展方向

当前业界对 GraphRAG 的探索大体有两条路线：一条强调社区摘要、全局主题感知和高层图摘要；另一条强调实体、关系与原文证据的细粒度联动。对于专业文档问答而言，后一条路线更适合需要页码、章节和具体证据出处的场景。
ragent 选择了细粒度证据图路线：图谱并不是替代原文，而是帮助系统在实体、关系和 chunk 之间组织更稳定的检索路径。

3.2 多模态文档解析与结构化索引进展

随着 MinerU 等工具成熟，PDF 不再只是 OCR 文本流，而是可以转成 Markdown、JSON 和内容块索引的结构化对象。真正的系统价值则来自二次工程化：如何把图片说明、表格行、页码和章节路径重新并入索引，使后续检索既保留结构，又能被大模型消费。
ragent 在这一步没有停留在“拿解析结果做 chunk”，而是继续建立 source_ref、section_path、page_number_start/end 等字段，为后续证据追溯做准备。

3.3 可解释问答与工作台化趋势

单一问答接口已经难以支撑复杂知识应用的调试与验证。越来越多系统开始强调推理链路展示、答案高亮、证据面板、图谱浏览、会话回放和视图保存等工作台能力。
kg-workbench 的价值就在于把 ragent 的 trace、图谱和 chunk 结果重新索引成本地工作台模型，使开发者和业务人员可以直接浏览图、证据和答案之间的对应关系。

表格 1 ragent 与 kg-workbench 的关键结构升级

层次	起点形态	现阶段实现	关键模块	改进价值
文档导入	简单文本入库	两阶段 MinerU 解析、状态机、分组处理、失败回滚	integrations.py / ragent.py	把 PDF、Markdown、图片和宽表纳入统一 ingestion pipeline
证据组织	chunk 为主	Entity-Relation-Chunk 异构证据图	operate.py	实体、关系和来源 chunk 可双向追溯
检索策略	单路向量召回	query variant + 图谱扩散 + 向量/图谱融合 + rerank + 覆盖选择	operate.py / rerank.py	适配复合问题、排序题和多依据题
数据形态	正文文本为主	图片说明、表格行级切分、宽表	integrations.py / wide_table.py	把非纯文本证据纳入统一检索空

All rights reserved 版权所有，侵权必究

		导入、chunk_text embedding 回填		间
上层交互	命令行/接口输出	图谱浏览、证据面板、QA trace、会话与视图管理	apps/api / apps/web	形成可解释问答与可视化分析工作台

4 算法设计与实现

4.1 关键技术与创新

从代码实现看，两个仓库并不是“算法仓库 + 前端仓库”的松散组合，而是以证据结构化为中心的上下层系统。ragent 负责把文档解析成可检索、可合并、可追溯的证据网络；kg-workbench 负责把图谱、chunk、文档、图片和 trace 重新组织成可浏览的本地工作台模型。

这种划分的好处是：底层引擎可以持续演进检索与问答能力，而上层工作台无需复写算法，只需桥接并展示结构化结果。

关键技术包括：

基于 MinerU 的两阶段专业文档解析与增强 Markdown 构建。

基于 chunk_text 证据节点的 Entity-Relation-Chunk 异构证据图。

多路混合检索：query variant、向量召回、图谱扩散、rerank 与证据覆盖选择。

工作台级可解释性：图谱浏览、证据追溯、问答链路展示与会话持久化。

4.2 算法原理及技术实现

4.2.1 底层结构化 ingestion pipeline

ragent/ragent.py 已经从早期的简单插入/查询入口演进为完整的数据入库编排器。其核心变化包括：引入 DocStatusStorage 管理 PENDING、PROCESSING、PROCESSED、FAILED 状态；按 source group 分组处理分段文档；对 staging chunk、vector upsert 和图谱合并进行并发控制；在组内失败时执行回滚；在退出时统一关闭异步 worker 与存储资源。与此同时，ragent 还新增了宽表导入分支，把 CSV、TSV、XLSX 等宽表记录转成行级实体与 chunk。

integrations.py 则把文档处理拆成“构建增强 Markdown”和“基于 Markdown AST 入索引”两阶段：前者通过 MinerU 生成 Markdown、content_list.json 和图片素材，再注入图片说明；后者通过 markdown-it 解析 AST，按章节路径、页码和表格结构切成可追溯 insert units，并把 source_ref、section_path、page_number_start/end 等元数据回填到 chunk。

4.2.2 Entity-Relation-Chunk 异构证据图与混合检索

operate.py 是整个项目的算法核心。实体关系抽取不再只是生成抽象的 entity 和 edge，而是把每个原文 chunk 同步建成 entity_type=chunk_text 的证据节点，并为实体、关系与 chunk 建立来源边，从而形成 Entity-Relation-Chunk 异构证据图。节点和边合并时会保留 source_chunk_ids、file_path、description、keywords 等字段；缺失的 chunk_text embedding 还会被自动回填，保证图谱节点可以再次进入向量检索。

查询阶段，系统会先做 query variant 拆解，再分别执行 chunk 向量召回、entity 向量召回、relationship 向量召回和图谱邻域扩散，随后将图谱支持度、matched_query_variants、relation support 与向量得分进行联合融合，必要时调用 rerank，再通过 source diversity、low-signal filtering、atomic variant coverage 和 token budget 控制选出最终上下文。tests/test_diversified_graph_retrieval.py 的 8 个测试用例正是围绕这一新检索链路编写的。

4.2.3 kg-workbench 可视化与可解释问答工作台

All rights reserved 版权所有，侵权必究

kg-workbench 的后端以 FastAPI + SQLite 为核心：parser.py 负责解析 ragent 输出的 GraphML、kv_store_text_chunks.json、kv_store_full_docs.json、Markdown、content_list 和图片文件；importer.py 负责把这些对象写入 nodes、edges、chunks、docs、sections、images、search_documents 等本地表，并提供 graph、subgraph、detail、search、view preset 和 QA session 接口。
qa.py 则把 ragent 的 one-hop / multi-hop trace 结果映射回本地工作台对象，恢复 node、edge、chunk、doc、section 与 image 的高亮集合；前端基于 React + Cytoscape 构建 GraphCanvas、EvidencePanel、DocumentPane 和 QuestionAnswerPanel，使用户不仅能看答案，还能查看“关键词提取 -> 图谱命中 -> 混合召回 -> rerank -> 最终证据”的完整链路。此次任务还将默认 QA 桥接根路径从 sibling ragent_master 调整为当前 /Users/liguowei/ubuntu/ragent，以保持工作台默认配置与当前仓库一致。

5 实验结果及解读

5.1 实验条件与数据准备

本次报告的实验与验证以两个本地仓库的真实代码、提交历史、benchmark 文件、trace 输出和界面截图为依据，不使用脱离代码的演示材料。ragent 的技术验证覆盖单元测试、核心模块阅读、benchmark 文档和样例 trace；kg-workbench 的验证覆盖前端 TypeScript 校验、后端 compileall、README 与 QA 桥接配置检查，以及界面截图审阅。
模型依赖：ragent 在 pyproject.toml 中引入 mineru[pipeline,vlm]==3.0.8、markdown-it-py、pandas/openpyxl、nano-vectordb、networkx、litellm 等依赖，并通过 ragent/llm/openai.py 与 ragent/rerank.py 兼容多 provider LLM、embedding 与 rerank 接口。
环境依赖配置：本次实际命令验证中，ragent 运行 python3 -m pytest tests/test_diversified_graph_retrieval.py，kg-workbench 运行 npm --workspace apps/web run lint 与 npm run check:api；同时将 kg-workbench 默认 QA 桥接根路径改为 /Users/liguowei/ubuntu/ragent，并保留 KG_WORKBENCH_RAGENT_ROOT 与 KG_WORKBENCH_RAGENT_PYTHON 的覆盖能力。

5.2 测试数据

5.2.1 代码与提交演进样本

从提交历史可以清晰看到 ragent 的升级路线：早期通过 refactor consolidate core modules and integrations 统一核心模块；随后加入 LiteLLM provider、宽表导入、chunk_text embedding backfill 和 MinerU 3.0.8；再继续增强 chunking and ingestion pipeline、keyword extraction and trace output、hybrid graph chunk selection，以及 diversify graph query context selection。kg-workbench 的主功能则在初始提交中完成，后续提交集中在 README、截图和最近的 feat(qa): render answer image references 等交互增强。

5.2.2 benchmark、trace 与样例项目

ragent 仓库内已有多组真实评测材料。benchmark/0403.kg_5.brief.eval.md 记录了一轮七题对比评审，其中 B 组平均总分为 8.09，对比 A 组 7.09，胜出 5 题；benchmark/benchmark_review.md 则记录了另一轮评审，A 组平均总分 8.06，略高于 B 组 7.99。两组结果共同说明：系统并非一次性定型，而是在多轮检索与答案生成策略试验中迭代优化。与此同时，kg-workbench/example/成人高血压食养指南_kg/model_usage_onehop_20260330_201748.md 显示单跳 trace 至少包含 embedding、rerank 与 chat 三类调用，共 8 次模型调用、20607 个总 token，能直接证明检索链

All rights reserved 版权所有，侵权必究

路的多阶段性。

ragent/example/demo_diet_kg_5/model_usage_onehop_20260412_152746.md 还展示了另一条更轻量的 one-hop 记录：14 次总调用中包含 11 次 embedding、1 次 rerank 与 2 次 chat，总 token 为 5018。这些真实输出可为报告中的 trace 论证提供样例支撑。

5.2.3 工作台截图与交互效果

kg-workbench/assets 中提供了两张真实界面截图。一张展示了图谱画布、图谱问答面板与右侧详情卡片的全局布局；另一张展示了图谱、证据面板、文档区和推理链路结果在同一页面中的联动关系。它们直接说明 kg-workbench 不是静态图谱浏览器，而是一个把问答结果、引用 chunk、相关边和原始文档片段统一串联起来的交互工作台。

All rights reserved 版权所有，侵权必究

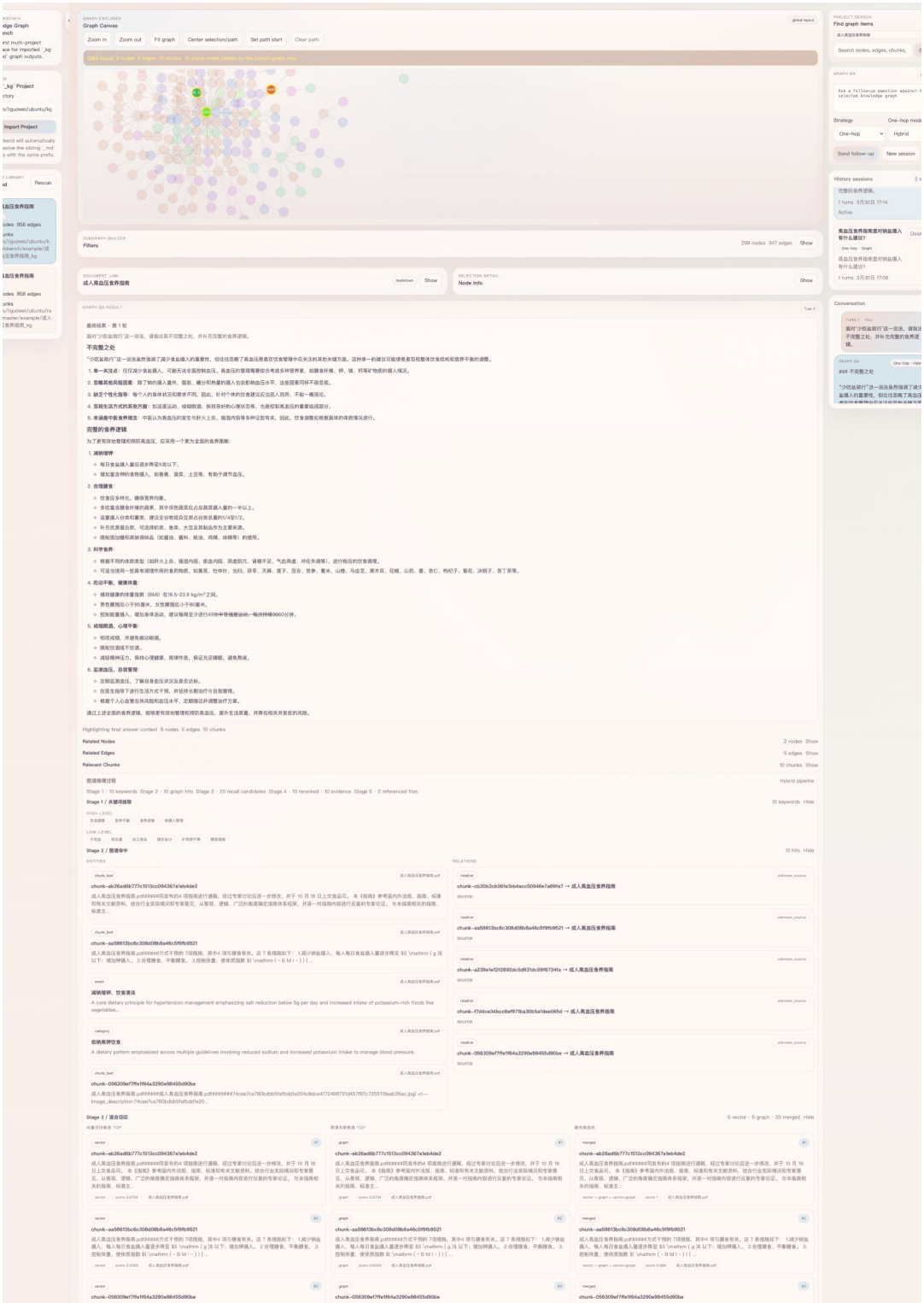

图 1 kg-workbench 全局图谱与问答工作台主界面

All rights reserved 版权所有，侵权必究

图 2 kg-workbench 图谱、证据面板与推理链路联动界面

表格 2 代码验证、benchmark 与 trace 实证摘要

证据类型	来源文件/命令	关键结果	说明
单元测	python3 -m pytest tests/test_diversified_graph_	8 passed	混合召回与证据覆盖选择相关逻辑通过

All rights reserved 版权所有，侵权必究

试	retrieval.py	in 6.81s	
前端校验	npm --workspace apps/web run lint	tsc --noEmit 通过	工作台前端类型检查通过
后端校验	npm run check:api	compileall 通过	QA 路径调整后 API 模块可编译
benchmark	0403.kg_5.brief.eval.md	B 组 5/7 胜，8.09 vs 7.09	最新一轮评测显示 KG-RAG 路线具备明显优势
trace 样例	model_usage_onehop_20260330_201748.md	8 次调用，20607 tokens	显示 embedding、rerank、chat 组合链路真实存在

5.3 测试步骤及结果

5.3.1 代码验证结果

实际命令执行结果表明：ragent 当前仓库内的 tests/test_diversified_graph_retrieval.py 共 8 个测试全部通过，覆盖 query 拆解、source_ref grouping、低信号过滤和最终 chunk 选择等关键行为；kg-workbench 的前端 TypeScript 校验与后端 compileall 在路径调整后也均通过。这意味着报告中关于代码正确性的判断不仅来自静态阅读，还经过了最小可复验的命令验证。

5.3.2 结构化检索与问答效果分析

从 benchmark 与 trace 可以看到，ragent 的改进不只体现在“是否答对”，还体现在“为什么答对”。benchmark 文档中多次出现的优点包括：证据与结论更匹配、对阈值与数值判断更稳定、回答更聚焦主问题；对应地，系统也暴露出不同方案间的权衡，如有的版本更全面但超出处证据，有的版本更稳健但展开略保守。这样的评测记录恰好说明 ragent 已经进入面向检索链路细节调优的阶段，而不是停留在单一 prompt 实验。

配合 model_usage_onehop 记录可以看到，系统在一次真实问题处理中会先做 embedding 与关键词相关步骤，再执行 rerank，最后进入答案生成阶段。kg-workbench 把这些 trace 字段进一步映射为图谱命中、merged candidates、rerank output 和 final context chunks，使系统的“中间证据层”第一次成为可视对象。

5.3.3 工作台联动展示效果

从图 1 和图 2 可以观察到，kg-workbench 的工程扩展主要体现在三方面：第一，图谱画布支持按节点类型、中心节点、深度和最小度数过滤子图；第二，问答面板可以展示答案、会话历史、分解步骤、推理链路和引用文件；第三，证据面板和文档面板会根据当前答案高亮具体 chunk、section 和 image。这种联动意味着 ragent 的底层结构化证据输出已经被完整吸收到上层产品中。

5.4 结论

综合代码、提交演进、命令验证、benchmark、trace 和截图，本项目已经形成清晰的上下层分工：ragent 作为底层结构化 KG-RAG 引擎，完成专业文档解析、异构证据图构建、混合检索、重排序与证据覆盖选择；kg-workbench 作为上层可视化与交互工作台，负责导入、浏览、检索链路回放、

All rights reserved 版权所有，侵权必究

证据追溯与问答会话管理。它们共同把系统从传统 chunk 检索 RAG 推进到“专业文档知识图谱构建、结构化证据检索、可解释问答和可视化分析”的完整形态。后续若继续增强，可沿更系统的 benchmark、更多图表目录样例和自动化报告生成链路展开。

6 引用

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401, 2020.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.

Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, et al. MinerU: An Open-Source Solution for Precise Document Content Extraction. arXiv:2409.18839, 2024.

opendatalab/MinerU official repository: https://github.com/opendatalab/MinerU .

本报告涉及的内部实现证据来自 ragent 与 kg-workbench 两个本地仓库的 README、pyproject/package 配置、benchmark 文档、example trace 文件及提交历史。

7 附录

7.1 关键模块与证据清单

7.1.1关键代码、命令与输出文件

ragent 的关键模块包括 ragent/ragent.py、ragent/operate.py、integrations.py、ragent/wide_table.py、ragent/rerank.py；kg-workbench 的关键模块包括 apps/api/app/parser.py、apps/api/app/importer.py、apps/api/app/qa.py、apps/web/src/components/GraphCanvas.tsx 和 apps/web/src/components/QuestionAnswerPanel.tsx。实际核验命令包括 python3 -m pytest tests/test_diversified_graph_retrieval.py、npm --workspace apps/web run lint、npm run check:api。最终输出文件为 /Users/liguowei/ubuntu/ragent/项目改进报告.docx。

All rights reserved 版权所有，侵权必究